

Hybrid-INoT без компрессии: аудит данных и факторный протокол ролевой генерации кода

Даниил Привезенцев^{1*}

^{1*} Национальный исследовательский университет «Высшая школа экономики», Москва, Россия.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Аннотация

Объединение нескольких ролей языковой модели в одном вызове уменьшает повторную передачу контекста, однако это преимущество учёта затрат не доказывает, что ролевые инструкции улучшают качество программ. В работе пересмотрена доказательная база Hybrid-INoT и предложено исследование, разделяющее эти вопросы. Повторный расчёт 240 сохранённых записей Flash-Lite подтверждает снижение токенового расхода пакета B3, включающего объединение вызовов и частичное сжатие контекста, на 49.6–90.3% относительно трёхвызовного B2. Вместе с тем при целевых контекстах 256, 1024 и 4096 токенов B3 расходует соответственно на 85.0, 37.5 и 10.3% больше токенов, чем простой одновызовный B0. Все сохранённые метки успешности равны единице, а полные решения отсутствуют. Эти данные не устанавливают ни пользу ролей, ни неухудшение качества. Новый дизайн пересекает один или три вызова с нейтральными или ролевыми инструкциями; полный предоставленный контекст одинаков во всех ячейках, компрессия отключена. Протокол предусматривает независимые выборки настройки и подтверждения, несколько seed, официальную проверку исполнением, воспроизведение внешних методов, анализ на уровне задач и полные архивы ответов. Основная цель — 1100 отложенных задач BigCodeBench после выделения 40 задач для настройки; исправление репозитория оценивается отдельно на SWE-bench. Это план: новых подтверждающих API-генераций и официальных результатов бенчмарков в данной версии нет. Вклад состоит в исправленной интерпретации данных и проверяемом протоколе определения пользы внутренних ролей сверх экономии от сокращения числа вызовов.

Keywords: Программная инженерия, языковые модели, факторный эксперимент, ролевые инструкции, токеновый бюджет, воспроизводимость, BigCodeBench, SWE-bench

1 Введение

Ассистент разработчика может спланировать изменение, реализовать его и проверить полученный результат. Эти операции можно поместить в один ответ модели или распределить между отдельными вызовами. Их также можно описать профессиональными ролями либо обычными последовательными инструкциями. Если одновременно изменить обе характеристики, большой наблюдаемый эффект становится трудно объяснить. Повторная передача кода увеличивает входной расход; критика может улучшать решение; ограничение длины способно помешать завершению реализации. Это разные механизмы.

В исходном исследовании Hybrid-INoT трёхвызовный конвейер planner-worker-critic сравнивался с объединённым ролевым вызовом, пороговым отбором контекста и возможным повторным вызовом. Наиболее сильное подтверждённое наблюдение относилось к расходу всего пакета. Исследование не установило, что экономию вызывают именно роли, что они повышают качество или что результат переносится на исправление реальных репозиториях. В новой версии утверждение об архитектурном превосходстве заменено конкретным вопросом: *какой вклад дают ролевые инструкции и границы вызовов при контроле предоставленной информации и выходного бюджета?*

Практическая значимость вопроса связана с выбором точки отсчёта. Сложный baseline способен сделать промежуточную систему привлекательной, даже если простой решатель дешевле. Повторный анализ V0 обнаруживает именно такую ситуацию: при первых трёх целевых длинах V3 требует больше токенов без улучшения сохранённых меток качества. Это наблюдение относится к малой выборке с потолочным качеством, но оно меняет мотивацию следующего эксперимента.

1.1 Вклад и статус доказательств

Текущая версия содержит три ограниченных вклада. Во-первых, арифметический аудит всех 240 сохранённых записей Flash-Lite с включением ранее недостаточно обсуждавшегося простого baseline. Во-вторых, явное тождество учёта затрат для трёхэтапного протокола без компрессии, отделённое от утверждений о качестве. В-третьих, факторный протокол с независимыми задачами настройки, наблюдаемыми воздействиями, полными архивами и официальной оценкой.

Рукопись представляет собой **аудит доказательств и перспективный протокол исследования**. Скачанные задачи, подготовленные манифесты запусков и программные тесты являются подготовительными артефактами, а не наблюдениями о качестве моделей. В данной версии не представлены новые масштабные сравнения, официальный результат SWE-bench, воспроизведение оригинального INoT или превосходство над внешними агентами. Маршрутизатор и политика проверки малой моделью исключены из основного метода: для них нужны самостоятельные экспериментальные воздействия.

2 Связанные исследования и открытый вопрос

2.1 Внутренние роли уже существуют в литературе

INoT Sun и Zeng организует внутреннее обсуждение с помощью программно-подобного промпта [1]. Следовательно, наличие нескольких ролевых имён в одном ответе не является новым архитектурным примитивом. Содержательный вопрос состоит в преимуществе конкретной реализации относительно равно структурированных нейтральных инструкций и исходного метода. Кроме того, в опубликованной версии INoT v1 есть неоднозначность остановки: в условии цикла используется логическая дизъюнкция, тогда как текстовое описание ограниченного числа итераций предполагает другую интерпретацию. Воспроизведение должно раскрывать выбранную трактовку; незаметно исправленный или значительно сокращённый промпт является адаптацией.

Выведенный план или критика — наблюдаемый текст, а не доказательство существования независимых скрытых агентов. Изучаемое воздействие относится к инструкциям и границам API-вызовов. Выводы о внутренней когнитивной сегрегации требуют дополнительных доказательств, которых не дают обычные метрики качества и расхода.

2.2 Контроль бюджета и практические методы сравнения

Tran и Kiela сравнивают одно- и мультиагентные рассуждения при одинаковом бюджете thinking-токенов [2]. Их задачи относятся к многошаговым рассуждениям, поэтому результаты нельзя переносить на генерацию программ. Однако работа показывает необходимость различать увеличение числа вызовов и увеличение вычислительного бюджета. В нашем исследовании отдельно рассматриваются одинаковый потолок выходных токенов, фактический расход, повторная передача входа, денежная стоимость и задержка.

Agentless предоставляет опубликованный конвейер локализации, исправления и проверки [3], а SWE-agent — интерфейс взаимодействия модели с репозиторием и инструментами [4]. Это необходимые внешние системные baseline. Самостоятельно написанный конвейер из трёх ролей не является ни одним из этих методов. Их поиск файлов и проверка промежуточных решений отличаются от генерации при фиксированном контексте; поэтому причинный факторный эксперимент и практическое сравнение систем должны проводиться отдельно. Показатели из таблиц лидеров нельзя включать в парный анализ с другими моделями или задачами.

2.3 Бенчмарки и достоверность проверки

BigCodeBench содержит 1140 задач программирования с использованием библиотек и подробных требований [5]. Это более широкий набор, чем несколько коротких функций HumanEval. Тем не менее его сложность для выбранных моделей необходимо измерить на выборке настройки. SWE-bench проверяет патчи тестами соответствующих репозитория [6, 7]; приближённая функция `check()` не измеряет исправление issue.

Официальная инфраструктура повышает сопоставимость, но не устраняет все угрозы валидности. Более поздний аудит SWE-bench Verified указывает на дефекты тестов и загрязнение обучающими данными при оценке сильнейших моделей [8]. Поэтому Verified используется для сопоставимого исследования репозитория, ошибки инфраструктуры отделяются от ошибок моделей, а для широких выводов о современной разработке требуется независимая проверка на более новых задачах. Само название сложного бенчмарка и официальный evaluator не доказывают внешнюю валидность.

3 Аудит исторических данных

3.1 Состав и проверяемость артефактов

Историческая реализация привязана к commit 010211ee5775185e8330ab6cde06e912c2adcb9e [10]. Репозиторий статьи содержит JSON эксперимента Flash-Lite E3 и предыдущий evidence snapshot [11]. В новой ревизии независимо пересчитаны агрегаты, проверена уникальность ключей задача-архитектура-seed и согласованность общего расхода с входными/выходными счётчиками и трассами отдельных вызовов. SHA-256 исходного файла и итоговая таблица сохранены вместе с анализом.

В наборе 20 уникальных задач HumanEval, четыре целевые длины контекста, три архитектуры и один seed: всего 240 записей. Это не 240 независимых задач. Рго-пилот использует пять задач и также имеет разведочный статус. Дополнительный контекст составлен из материалов других задач и не представляет измеренный набор необходимых зависимостей репозитория. Целевая длина является меткой условия, а не универсальной оценкой размера кода.

В 240 записях Flash-Lite отсутствуют полные итоговые решения и исходные ответы API. Можно проверить арифметику, но нельзя повторно исполнить именно те решения, которым назначены метки pass/fail. Генерация ответа заново создаст новое наблюдение и не восстановит утраченный артефакт. Поэтому далее указаны *сохранённые метки успеха*, а не заново подтверждённая корректность.

Трассы расхода выявляют ещё одно уточнение: из 80 записей V3 только 32 содержат один вызов, а 48 включают дополнительный вызов с меткой **rerun**. Поэтому V3 в среднем использует 1.6 вызова при каждой длине контекста, тогда как V0 и V2 содержат ровно один и три вызова. Итоговый расход V3 учитывает эти повторы. Его конечные метки успеха нельзя интерпретировать как pass@1 одной попытки; историческое сравнение не является чистым контрастом трёх и одного вызова даже до учёта компрессии.

3.2 Простой baseline меняет интерпретацию

Таблица 1 содержит средний суммарный расход, непосредственно пересчитанный из сохранённого файла. V0 — один решатель; V2 — трёхвызовный конвейер; V3 — объединение ролей с пороговым отбором контекста. Все сохранённые метки успешности равны единице.

Таблица 1 Арифметический аудит Flash-Lite. В каждой строке те же 20 задач, seed 42. Проценты рассчитаны из средних расходов; это не среднее индивидуальных процентов. Полные ответы отсутствуют.

Контекст	B0 токены	B2 токены	B3 токены	B3 к B2 экономия (%)	B3 к B0 изменение (%)
256	745.65	2737.90	1379.25	49.62	+84.97
1024	1650.80	5486.80	2270.50	58.62	+37.54
4096	5133.70	16006.75	5663.55	64.62	+10.32
16384	19159.50	58110.00	5632.90	90.31	−70.60

Из таблицы следуют три вывода. Во-первых, экономия пакета относительно B2 арифметически воспроизводится. Во-вторых, при первых трёх целевых длинах B3 дороже B0; пользу ролевых инструкций из потолочных меток качества установить нельзя. В-третьих, изменение соотношения при 16384 токенах совпадает с отбором контекста в B3 при полной передаче в B0/B2. Это не доказывает, что сами роли становятся выгоднее на длинном входе.

Сохранённые метаданные контекста также различают целевые метки и фактическое преобразование. Отношение размера преобразованного контекста к исходному равно единице при целях 256 и 1024, но лежит в диапазоне 0.932–1.005 при цели 4096 и 0.242–0.251 при цели 16384. Поэтому условие 4096 не является полностью несжатым контролем. Это сохранённые отношения размеров из реализации, а не независимая повторная токенизация запросов.

Почти горизонтальная зависимость B3 выше порога согласуется с ограничением размера отобранного контекста. Прямая, построенная через порог, смешивает разные режимы работы. Её наклон характеризует четыре точки дизайна, но не предельную стоимость архитектуры при неизменных остальных механизмах. Поэтому такой наклон исключён из главного результата.

3.3 Почему сохранение качества не установлено

Малые p -значения токенных различий не доказывают сохранение качества. При потолке нет наблюдаемой вариации, позволяющей выяснить, помогают или мешают роли на сложных задачах. В идеализированной модели независимых биномиальных наблюдений ноль вредных исходов среди $n = 20$ пар даёт одностороннюю верхнюю 95%-ную границу

$$u = 1 - 0.05^{1/20} = 0.1391. \quad (1)$$

Это 13.91 процентного пункта, существенно больше прежнего допуска в два пункта. Фактическая выборка первых k задач не является iid-выборкой; расчёт служит ориентиром планирования, а не гарантией для популяции. Повторение тех же задач на четырёх длинах не увеличивает число независимых единиц в четыре раза.

Предыдущая проверка малой моделью сохранила агрегаты, а не полные решения по случаям и результаты повторных запусков. Согласие на приближённом SWE-bench, включая показатель 0.30, не является официальной долей исправленных issue. Причинный эффект screening на стоимость повторов не оценён. Маршрутизация и параллельные инструменты также не устраняют проблему атрибуции в E3 и исключаются из текущего семейства гипотез.

4 Экспериментальное воздействие без компрессии

4.1 Два фактора при одинаковой информации

Новый дизайн пересекает топологию вызовов $t \in \{s, m\}$, где s означает один вызов, m — несколько, с ролевой организацией $r \in \{0, 1\}$. Все четыре ячейки получают одинаковую постановку и полный предоставленный контекст. В нейтральном и ролевом режимах совпадают операции планирования, реализации и проверки; различаются назначения профессиональных ролевых имён.

Таблица 2 Основные факторные условия. Во всех ячейках отсутствуют компрессия, маршрутизация и обратная связь от evaluator во время генерации.

Условие	Вызовы	Имена	Исполнение
Один, нейтральный	1	нейтральные	Три операции в одном ответе
Один, ролевой	1	роли	Те же операции в одном ответе
Три, нейтральный	3	нейтральные	По операции на вызов, полная история
Три, ролевой	3	роли	Те же отдельные операции и история

Предложенный рецензентом дизайн «топология $\times$ компрессия» изолировал бы компонент сжатия исторического пакета. Автор требует исключить компрессию из нового основного эксперимента. Поэтому выбран дизайн «топология $\times$ роли», где сжатие отсутствует и непосредственно проверяется ролевой вклад. Он не разлагает задним числом исторический эффект; это принципиальное различие оцениваемых величин, а не переименование невыполненного опыта.

В многовызовных режимах каждый этап получает исходную постановку, контекст и все предыдущие ответы без усечения. В одновызовных три этапа разворачиваются внутри одного ответа. Таким образом, границы вызовов меняют и представление промежуточной информации. Оценивается определённый протокол, а не абстрактная топология, независимая от любого информационного потока. Полные промпты и правила выходного формата фиксируются до подтверждающих запусков.

4.2 Выходной бюджет и ограничения длины

Начальный общий потолок — 12288 выходных токенов на задачу и режим. Для трёх вызовов он делится поровну: по 4096 на этап; для одного вызова доступен целиком. Это равенство допустимого completion-бюджета, но не всех токенов, фактических вычислений, денег или времени. В начальном дизайне неиспользованный остаток этапа не переносится; ограничение раскрывается явно.

Такое распределение само способно ухудшать конвейер. Поэтому на задачах настройки проверяются причины завершения ответа и полнота итогового кода. Если хотя бы в одном режиме более 5% ответов упираются в ограничение длины, подтверждающая серия не начинается до фиксации нового бюджета или распределения. Дополнительный опыт с одинаковым потолком на этап проверяет чувствительность к бюджету, но имеет другую оцениваемую величину и не объединяется с основным без маркировки. Ограниченные по длине ответы остаются в своём знаменателе; вход нельзя незаметно сократить до лимита API.

4.3 Без адаптивного поиска внутри факторной генерации

Генератор создаёт один итоговый кандидат. Скрытые тесты исполняются после генерации и не передаются в следующий вызов. Три seed означают три отдельных попытки; успех хотя бы одной из трёх нельзя называть pass@1. Маршрутизатор, малая проверяющая модель, fallback и адаптивные повторы не выбирают выполняемую ячейку. Фиксированное назначение не зависит от тестовых исходов; практические системы с инструментами исследуются отдельно.

5 Что доказывает модель расходов

Пусть C — токенизированный общий вход, P_j — служебные инструкции этапа, o_j — его выход. Если предыдущие ответы передаются полностью, суммарный вход трёхвызовной реализации равен

$$T_m^{\text{in}} = 3C + \sum_{j=1}^3 P_j + 2o_1 + o_2. \quad (2)$$

С учётом выходов получаем

$$T_m = 3C + \sum_{j=1}^3 P_j + 3o_1 + 2o_2 + o_3. \quad (3)$$

Для объединённого вызова $T_s = C + P_s + o_s$, поэтому

$$T_m - T_s = 2C + \sum_j P_j - P_s + 3o_1 + 2o_2 + o_3 - o_s. \quad (4)$$

Это тождества учёта для определённого формата сообщений, где служебные токены включены в P . Они не требуют компрессии. Имена ролей не входят в

коэффициент повторной передачи контекста. Их возможная польза состоит в изменении качества или выходного поведения и требует измерения.

Уравнение (4) не гарантирует положительную разность: длины выходов могут различаться. Токенная разность также не тождественна денежной: имеют значение кэширование, тариф модели/провайдера и отдельные услуги. Нужно сохранять возвращённые API счётчики входа, чтения кэша, выхода и доступных reasoning-токенов [9]. Reasoning может уже входить в completion; двойной учёт недопустим. Основная денежная метрика — сообщённая провайдером стоимость в USD; пересчёт по единому тарифу является отдельным анализом чувствительности.

Стоимость успешно выполненной задачи оценивается отношением всех затрат к общему числу успехов, а не средним расходом только успешных случаев. При нуле успехов отношение не определено. API-расход не включает настройку среды, тестирование, хранение и ручную проверку. Экстраполяция в совокупную стоимость владения в этой статье не выполняется.

6 План новых экспериментов

6.1 Выборка и независимые единицы

Скачанный релиз BigCodeBench v0.1.4 содержит 1140 задач; версия и SHA-256 записаны в репозитории. Идентификаторы сортируются, случайно переставляются с seed 20260908; 40 задач выделяются для настройки, 1100 остаются для подтверждения. Разбиение не использует исходы. Три seed генерации (17, 43, 101), четыре режима и две модели дают 26400 планируемых генераций и 52800 вызовов. Это целевая матрица; новых подтверждающих генераций в текущей версии ноль.

Выборка настройки служит для проверки API и evaluator, выходного бюджета, параметров провайдера и ресурсов. Она исключена из подтверждающих оценок. Поправки фиксируются до просмотра отложенных исходов. Опубликованный Hard-поднабор может стать заранее определённой стратой, но не новым независимым набором. Единицей обобщения остаётся задача, а не строка результатов или повтор генерации.

Начальные экономичные кандидаты — Qwen3 Coder 30B A3B Instruct и Gemini 2.5 Flash-Lite. Их API-идентификаторы и объявленные тарифы проверены 8 сентября 2026 года. Алиас API не является неизменным снимком весов. До платного пробного вызова фиксируется провайдер, поддерживающий все параметры; fallback отключается, фактически возвращённые идентификаторы сохраняются. Поддержка seed не объявляется гарантией детерминированного повторения.

6.2 Исправление репозитория исследуется отдельно

Вторая целевая серия — все 500 экземпляров SWE-bench Verified, три seed, четыре факторных режима и одна начальная модель: 6000 генераций и 12000 вызовов. До inference необходимо сформировать пакет файлов репозитория на `base_commit` по правилу поиска, не использующему исходы. Во всех режимах используется один

и тот же полный пакет. Поиск выбирает файлы, поэтому «без компрессии» означает сохранение предоставленного пакета, а не помещение произвольно большого репозитория в любое окно модели.

Эталонный патч, патч тестов, будущие коммиты и служебные сведения evaluator не входят в промпт. Подготовленный интерфейс генерации сам по себе не реализует этот поиск по репозиторию. Для выводов о полном процессе нужны одновременно найденный контекст, полученный патч, журнал применения и официальные тесты. Проверка генерации отдельной функцией не заменяет эту серию.

6.3 Официальные проверки и контрольные случаи

Решения BigCodeBench экспортируются в официальный формат и оцениваются в зафиксированной изолированной среде. Для SWE-bench сохраняются идентификатор задачи, имя модели и патч. Официальный harness применяет патч и создаёт отчёты по случаям [7]. До оценки модели необходимо исполнить эталонный контроль и заведомо неправильный контроль, затем проверить полученные тестовые отчёты. Успешное завершение процесса evaluator само по себе не означает успешного решения.

Сохраняются вывод тестов, result JSON, команды, версия исходников, digest образа/среды и хэш проверенного решения. Пустой или неправильный программный ответ является неуспехом модели. Ошибка сборки окружения является инфраструктурным исходом и учитывается отдельно. Новый идентификатор оценки, зависящий от байтов предсказаний, предотвращает использование кэшированного результата другого патча. Текущая версия не утверждает, что официальные контрольные или модельные проверки уже выполнены.

6.4 Точность воспроизведения baseline

Полное сравнение требует прямого одновызовного решателя, оригинального INoT, Agentless и SWE-agent сверх четырёх причинных ячеек. Прямой решатель показывает, нужны ли структурированные этапы вообще. Оригинальный INoT проверяет добавленную ценность относительно непосредственного предшественника. Внешние системы оценивают практическую значимость при настоящем поиске файлов и инструментах.

Для каждого метода фиксируются commit, промпт и конфигурация, модель/провайдер, задачи, изменённые бюджеты, полные траектории и официальные оценки. По возможности сохраняется исходное поведение. Если метод ищет кандидатов или запускает промежуточные тесты, все вызовы учитываются, а сравнение обозначается системным. Переименованный собственный конвейер, вольный пересказ INoT или чужой опубликованный процент не закрывают требование к baseline. Новые результаты этих сравнений в ревизии не заявлены.

6.5 Осуществимость и ограничение затрат

Автор установил максимум 300 USD OpenRouter на все этапы, повторы и возобновлённые запуски. Начальное распределение: 10 USD на настройку, 190 USD на

основную серию, 75 USD на пилоты репозитория/baseline и 25 USD резерва. Это потолки расходов, а не доказательство того, что все матрицы укладываются в бюджет. До фиксации подтверждающей серии нужно проверить консервативные оценки запросов и фактический расход пробных вызовов.

Если средств хватает только на часть задач, подвыборка выбирается независимо от исходов и описывается как меньшая разведочная серия. Запуск до исчерпания средств с последующим объявлением полученного префикса полным исследованием вернул бы смещение отбора. При начальной проверке ключ не имел доступного остатка; новых платных генераций на этом этапе не выполнено. Неопределённые списания резервируют весь допустимый расход запроса до сверки.

7 План статистического анализа

7.1 Оцениваемые величины и гипотезы

Пусть $Y_{i,t,r,k}$ — бинарный официальный тестовый исход для задачи i , топологии t , ролевого режима r и seed k . Среднее по фиксированным seed обозначим $\bar{Y}_{i,t,r}$. Эффект ролей внутри топологии, эффект топологии внутри ролевого условия и взаимодействие определяются как

$$\Delta_R(t) = \mathbb{E}_i[\bar{Y}_{i,t,1} - \bar{Y}_{i,t,0}], \quad (5)$$

$$\Delta_T(r) = \mathbb{E}_i[\bar{Y}_{i,s,r} - \bar{Y}_{i,m,r}], \quad (6)$$

$$\Gamma = \Delta_R(s) - \Delta_R(m). \quad (7)$$

Взаимодействие Γ показывает, зависит ли эффект ролевых инструкций от протокола вызовов. Один главный эффект этого не устанавливает. Для каждой модели и бенчмарка публикуются четыре средних и все пять контрастов. Аналогичные разности USD характеризуют стоимость; логарифмические контрасты вторичны и допустимы лишь при строго положительном расходе.

Гипотеза пользы ролей относится к $\Delta_R(s)$ и проверяется двусторонним тестом и интервалом: отрицательный или нулевой результат допустим. Ресурсная гипотеза сравнивает денежный расход одного ролевого вызова и трёх ролевых вызовов. Неухудшение качества проверяется отдельно:

$$H_0 : \Delta_T(1) \leq -0.02, \quad H_1 : \Delta_T(1) > -0.02. \quad (8)$$

Сохранение качества устанавливается лишь когда односторонняя нижняя 97.5%-ная граница выше -0.02 . Заявленный выигрыш «цена-качество» требует и ресурсного преимущества, и выполнения этого критерия. Незначимый тест превосходства не является доказательством неухудшения.

7.2 Зависимость, множественность и пропуски

Ресэмплируются задачи с сохранением всех их режимов, seed и парных моделей: 10000 bootstrap-повторов, seed 20260908. Seed — повторные измерения, а

не независимые задачи. Модели анализируются отдельно; объединённая оценка допустима только с явными весами. Для пяти качественных контрастов применяется поправка Холма внутри заранее определённого семейства бенчмарк/модель. Оценки и неопределённость публикуются и при незначимых скорректированных тестах.

При малом числе расходящихся исходов дополнительно приводятся таблица парных результатов для заранее выбранного seed 17 и консервативные точные границы. Для публикационного вывода должны быть выше -0.02 и bootstrap-граница, и граница первого seed. Последняя равна нижней границе Клоппера–Пирсона для полезных исходов минус верхняя граница для вредных; каждому хвосту выделяется вероятность 0.0125. Это не точный интервал среднего по нескольким seed. Bootstrap-интервал нулевой ширины на полностью успешном наборе не доказывает сохранение качества. Парные тесты случайной смены знаков приводятся как анализ чувствительности, предполагающий симметрию. Анализ может возвращать «неопределённо», даже если вся матрица заполнена.

Пустые ответы, некорректный код и неуспешные патчи считаются неудачами модели. Инфраструктурные исходы обозначаются пропусками; отдельно вычисляются оптимистические и пессимистические границы, где неразрешённым наблюдениям назначается успех или неуспех. Основной парный анализ полных задач требует наличия всех seed/режимов и раскрытия числа исключённых задач. Анализ только полных случаев не может незаметно заменить исходный знаменатель назначенных задач. Исключение по результату, настройка промптов на проверочной выборке и выборочная публикация seed запрещены.

7.3 Мощность: почему большой матрицы может быть мало

Для одного seed парная разность $D \in \{-1, 0, 1\}$ имеет около нулевого среднего дисперсию $\text{Var}(D) \approx d$, где d — вероятность расходящихся исходов. Нормальное приближение для 80%-ной мощности при двухпунктовом допуске и одностороннем $\alpha = 0.025$ даёт

$$n \approx \frac{(z_{0.975} + z_{0.80})^2 d}{0.02^2}. \quad (9)$$

Таблица 3 Приближённое требование к независимым задачам по уравнению (9). Это планирование, а не достигнутая мощность.

Вероятность расхождения d	0.05	0.10	0.20
Число задач с округлением вверх	982	1963	3925

Прежний расчёт 149 пар относился к благоприятному случаю односторонней 95%-ной биномиальной границы при нуле вредных исходов; это не дизайн неухудшения с мощностью 80%. Даже 1100 задач не гарантируют достаточной точности. До фиксации основной серии нужно симулировать зависимость повторных seed, используя только оценки с выборки настройки. Если бюджет

недостаточен, публикуются эффекты и неопределённость; менять допуск после просмотра результатов нельзя.

8 Воспроизводимость как цепочка доказательств

Для воспроизводимости недостаточно commit и хэша агрегированных чисел. Нужна связь между зафиксированным входом, точным API-запросом, возвращённым ответом, извлечённым кандидатом, выполненными тестами и строкой анализа. Хэш полезен только при наличии самого доступного артефакта.

До каждого вызова сохраняются запрос без заголовка авторизации и резерв его максимальной стоимости. Полное тело ответа, идентификаторы провайдера, использование API и причина завершения записываются до разбора. Возвращённый текст хранится полностью; доступ к нераскрываемым рассуждениям модели не заявляется. Промежуточные ответы сохраняются до следующего вызова, чтобы последующая ошибка не уничтожила предыдущие доказательства. Возобновление допустимо лишь при совпадении хэшей задач, настроек, промптов/исходников и манифеста и отсутствии несверенного списания.

Аудит архива проверяет ожидаемые сочетания задача–модель–режим–seed, дубликаты, целостность байтов, согласованность токенов/расходов и связь кандидата с тестами. Исторические файлы остаются отдельным набором; новые генерации в них не вставляются. Ошибка извлечения кода является наблюдаемым исходом, а не разрешением вручную выбрать более удачный фрагмент.

Реализация и версионизируемый протокол сопровождают рукопись в репозитории [11]. Unit tests и подготовительные проверки относятся к инвариантам программы; их нельзя включать в экспериментальную выборку или трактовать как успех модели на бенчмарке. Для эмпирического вывода о превосходстве всё ещё нужны реализация поиска по репозиторию, выполненные upstream-baseline и полные записи официального evaluator.

9 Интерпретация, ограничения и граница публикации

Исправленный исторический вывод конкретнее общего заявления об эффективности. Объединение вызовов с отбором контекста дешевле исторического трёхвызовного пакета. На целевых длинах без сжатия простой решатель дешевле ещё больше. Поэтому лучшая организация по соотношению затрат и качества остаётся открытым вопросом, а не заранее благоприятным для внутренних ролей результатом.

Новый факторный эксперимент оценивает эффекты фиксированных промптов и протокола на своей совокупности задач. Он не устанавливает независимость скрытых ролей, универсальную пользу на всех провайдерах, корректность сверх тестов или перенос на неизвестное распределение репозиториев. Равный completion-потолок оставляет разными вход, кэширование и реальные вычисления. Бюджеты этапов могут взаимодействовать со сложностью; фиксированный

контекст исключает адаптивное получение информации. Это основания для отдельных анализов чувствительности и практического сравнения систем.

Целевая выборка существенно больше исторической, но её размер пока является планом. Загрязнение обучающими данными, дефекты проверки, зависимость похожих задач и изменение провайдера сохраняют значение даже после полного исполнения. Для репозитория дополнительно нужны настоящий поиск файлов и рабочая изолированная среда. Отсутствие новых подтверждающих исходов в этой версии не позволяет объявить существенные эмпирические замечания уже закрытыми.

10 Заключение

Сохранённые данные подтверждают пакетную экономию относительно трёх вызовов, однако восстановленное сравнение с простым решателем ослабляет обоснование пользы самих ролей. Конфаундированное утверждение заменено вопросом без компрессии, явными тождествами затрат и протоколом, способным опровергнуть предполагаемое преимущество. Для эмпирической публикации необходимы полные ответы моделей, официальные тесты, содержательные baseline и неопределённость на уровне задач. Ревизия предоставляет исправленную интерпретацию и постановку; масштабный подтверждающий результат ещё предстоит получить.

Доступность данных и кода

Репозиторий статьи [11] содержит две языковые версии, PDF, исторические записи, арифметический аудит, хэши источников/набора и перспективный протокол. Историческая реализация зафиксирована отдельно [10]. Официальные наборы и evaluator сохраняют собственные лицензии. Ответы моделей и тестовые артефакты нового подтверждающего исследования пока отсутствуют, поскольку серия не выполнена.

Декларации

Финансирование и конфликт интересов: в предыдущей рукописи внешнее финансирование и конфликт интересов не заявлялись. **Предмет исследования:** публичные программные бенчмарки; исследование с участием людей не представлено. **Ответственность автора:** автор отвечает за итоговые научные утверждения и утверждение завершённой экспериментальной записи. При переработке рукописи и подготовке исследовательского ПО использовалась помощь ИИ; программные проверки отделены от экспериментальных результатов.

Список литературы

- [1] H. Sun and S. Zeng. Introspection of Thought Helps AI Agents. arXiv:2507.08664v1, 2025. <https://arxiv.org/abs/2507.08664v1>.

- [2] D. Tran and D. Kiela. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460v2, 2026. <https://arxiv.org/abs/2604.02460v2>.
- [3] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024. <https://arxiv.org/abs/2407.01489>.
- [4] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS, 2024. <https://arxiv.org/abs/2405.15793>.
- [5] T. Y. Zhuo et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. ICLR, 2025. <https://arxiv.org/abs/2406.15877>.
- [6] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. <https://arxiv.org/abs/2310.06770>.
- [7] SWE-bench maintainers. Evaluation Guide. Accessed 8 September 2026. <https://www.swebench.com/SWE-bench/guides/evaluation/>.
- [8] OpenAI. Why SWE-bench Verified no longer measures frontier coding capabilities. 2026. <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>.
- [9] OpenRouter. Models API and provider usage metadata. Snapshot accessed 8 September 2026. <https://openrouter.ai/api/v1/models>.
- [10] D. Privezentsev. INoT_Research. Historical implementation, commit 010211ee5775185e8330ab6cde06e912c2adcb9e. https://github.com/daniil-novel/INoT_Research.
- [11] D. Privezentsev. Article-INoT: manuscript, historical artifacts and compression-free revision protocol. 2026. <https://github.com/daniil-novel/article-INoT>.